Experiment 11: Unique Paths in a Grid

Aim:

To find the number of unique paths for a robot to travel from the top-left corner to the bottom-right corner of an m × n grid, moving only right or down.

Algorithm:

1. Read the number of rows m and columns n.
2. Create a 2D array dp of size m × n.
3. Initialize the first row and first column with 1.
4. For every remaining cell, calculate dp[i][j] = dp[i-1][j] + dp[i][j-1].
5. Return dp[m-1][n-1].

Code (Python):

```
def unique_paths(m, n):
    dp = [[1] * n for _ in range(m)]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
    return dp[m - 1][n - 1]

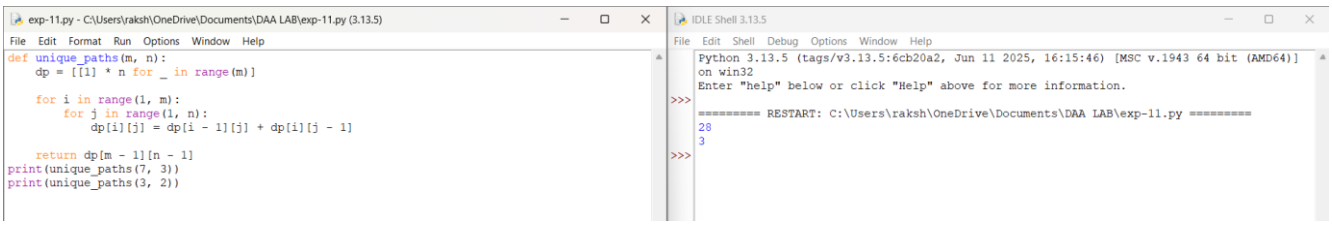
print(unique_paths(7, 3))
```

Input:

m = 7, n = 3

Output:

28



Time Complexity: O(mn)

Space Complexity: O(mn)

Result:

The number of unique paths from the top-left to the bottom-right corner was calculated successfully.

Experiment 12: Game of Life

Aim:

To calculate the next generation of a cellular grid using the rules of Conway's Game of Life.

Algorithm:

1. Read the current binary grid.
2. Count the live neighbors of each cell.
3. Apply the four rules of the Game of Life.
4. Update all cells simultaneously, using a copy of the original grid or temporary state markers.
5. Display the updated grid.

Code (Python):

```
def game_of_life(board):
    m, n = len(board), len(board[0])
    original = [row[:] for row in board]
    for i in range(m):
        for j in range(n):
            live = 0
            for di in (-1, 0, 1):
                for dj in (-1, 0, 1):
                    if di == 0 and dj == 0:
                        continue
                    r, c = i + di, j + dj
                    if 0 <= r < m and 0 <= c < n:
                        live += original[r][c]
            if original[i][j] == 1:
                board[i][j] = 1 if live in (2, 3) else 0
            else:
                board[i][j] = 1 if live == 3 else 0
    return board

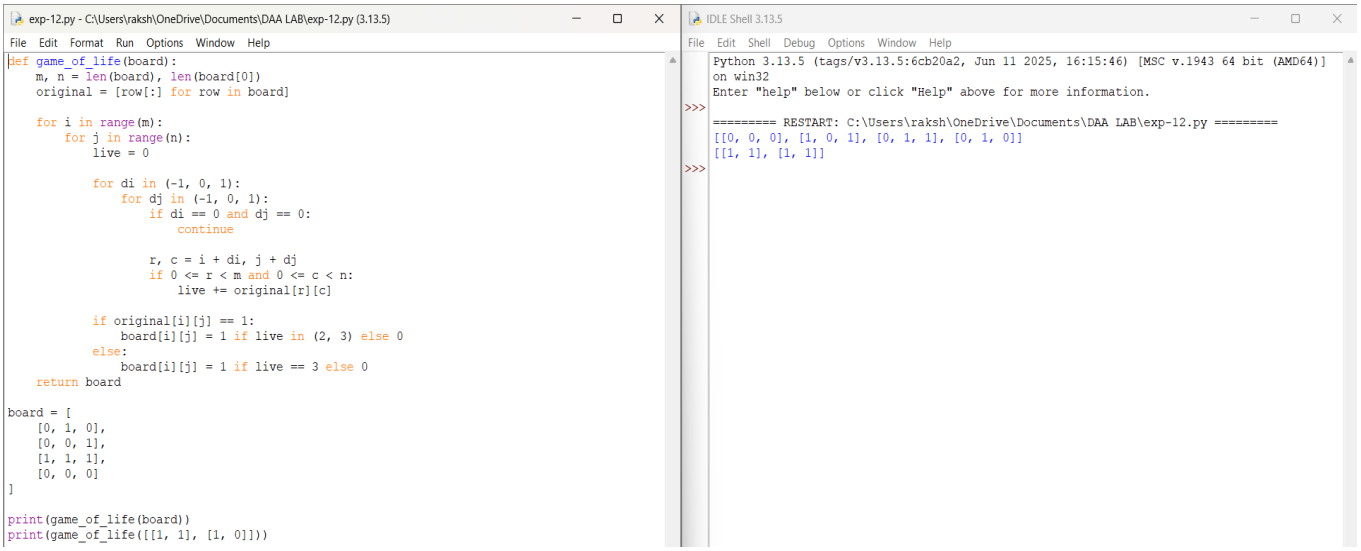
board = [
    [0, 1, 0],
    [0, 0, 1],
    [1, 1, 1],
    [0, 0, 0]
]
print(game_of_life(board))
```

Input:

[[0, 1, 0],
 [0, 0, 1],
 [1, 1, 1],
 [0, 0, 0]]

Output:

[[0, 0, 0],
 [1, 0, 1],
 [0, 1, 1],
 [0, 1, 0]]



Time Complexity: O(mn)

Space Complexity: O(mn)

Result:

The next generation of the grid was obtained successfully by applying the Game of Life rules.

Experiment 13: Champagne Tower

Aim:

To determine how full a particular glass is after pouring a specified number of cups of champagne into a tower.

Algorithm:

1. Read poured, query_row, and query_glass.
2. Create an array to store the amount of champagne reaching each glass.
3. Pour the specified amount into the top glass.
4. If a glass contains more than 1 cup, distribute half of its excess to each glass directly below it.
5. Repeat for each row until the required row is reached.
6. Return the queried glass's amount, limited to a maximum of 1.

Code (Python):

```
def champagne_tower(poured, query_row, query_glass):
    tower = [[0.0] * (i + 1) for i in range(query_row + 1)]
    tower[0][0] = float(poured)
    for i in range(query_row):
        for j in range(i + 1):
            excess = max(0.0, tower[i][j] - 1.0)
            if excess > 0:
                tower[i + 1][j] += excess / 2
                tower[i + 1][j + 1] += excess / 2
    return min(1.0, tower[query_row][query_glass])

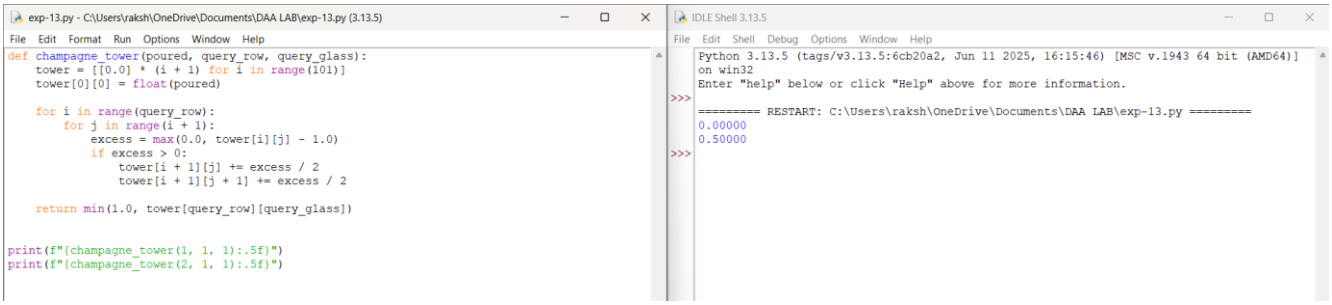

print(f"{champagne_tower(2, 1, 1):.5f}")
```

Input:

poured = 2
query_row = 1
query_glass = 1

Output:

0.50000



Time Complexity: $O(r^2)$

Space Complexity: $O(r^2)$

Result:

The amount of champagne in the specified glass was calculated successfully.

Experiment 14: Sorting Different Types of Lists

Aim:

To test list sorting with an empty list, a single element, identical elements, and negative numbers.

Algorithm:

1. Define a sorting function.
2. Read the input list.
3. Sort the elements in ascending order.
4. Display the sorted list.
5. Test the function using all four test cases.

Code (Python):

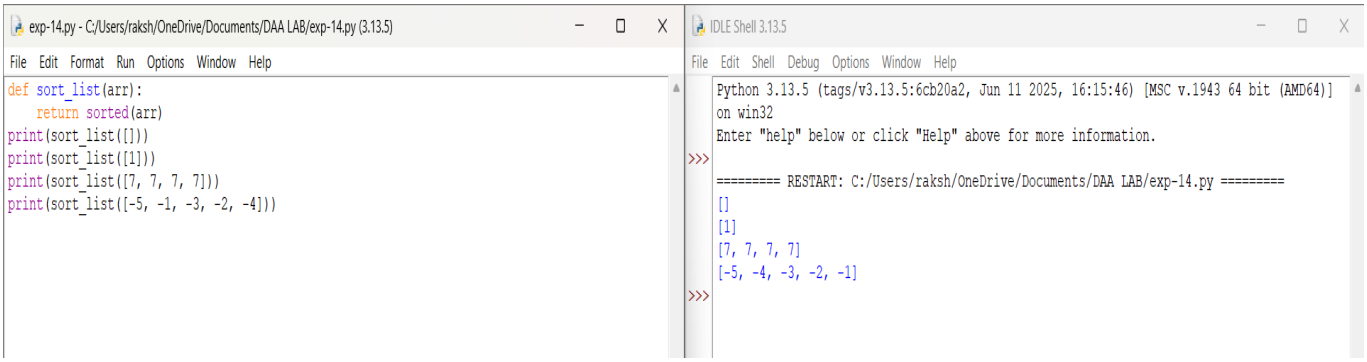
```
def sort_list(arr):
    return sorted(arr)
print(sort_list([]))
print(sort_list([1]))
print(sort_list([7, 7, 7, 7]))
print(sort_list([-5, -1, -3, -2, -4]))
```

Input:

```
[]
[1]
[7, 7, 7, 7]
[-5, -1, -3, -2, -4]
```

Output:

```
[]
[1]
[7, 7, 7, 7]
[-5, -4, -3, -2, -1]
```



Time Complexity: O(n log n)

Space Complexity: O(n)

Result:

The sorting function handled empty, single-element, identical-element, and negative-number lists successfully.

Experiment 15: Selection Sort

Aim:

To sort an array in ascending order using the Selection Sort algorithm.

Algorithm:

1. Read the input array.
2. Find the smallest element in the unsorted portion.
3. Swap it with the first element of the unsorted portion.
4. Move the boundary of the sorted portion one position to the right.
5. Repeat until the array is sorted.
6. Display the sorted array.

Code (Python):

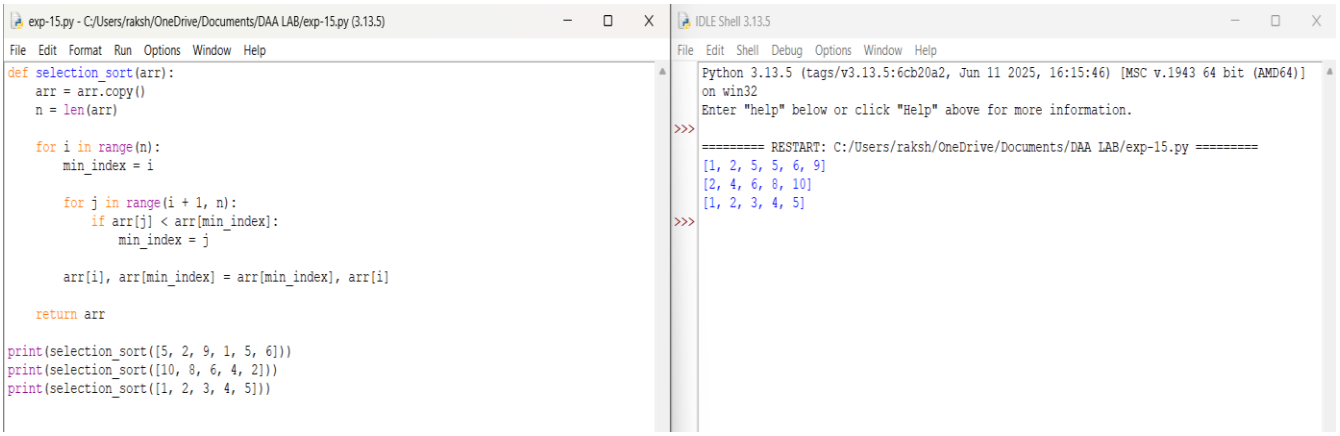
```
def selection_sort(arr):
    arr = arr.copy()
    n = len(arr)
    for i in range(n):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
print(selection_sort([5, 2, 9, 1, 5, 6]))
print(selection_sort([10, 8, 6, 4, 2]))
print(selection_sort([1, 2, 3, 4, 5]))
```

Input:

[5, 2, 9, 1, 5, 6]
[10, 8, 6, 4, 2]
[1, 2, 3, 4, 5]

Output:

[1, 2, 5, 5, 6, 9]
[2, 4, 6, 8, 10]
[1, 2, 3, 4, 5]



Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Result:

The input arrays were sorted successfully in ascending order using Selection Sort.

Experiment 18: Large Groups in a String

Aim:

To identify consecutive groups of three or more identical characters in a string.

Algorithm:

1. Read the input string.
2. Start at the current character and scan forward while characters remain identical.
3. If the group contains at least three characters, record its starting and ending indices.
4. Continue scanning from the end of the current group.
5. Display the list of index ranges.

Code (Python):

```
def large_groups(s):
    result = []
    i = 0
    while i < len(s):
        j = i
        while j < len(s) and s[j] == s[i]:
            j += 1
        if j - i >= 3:
            result.append([i, j - 1])
        i = j
    return result

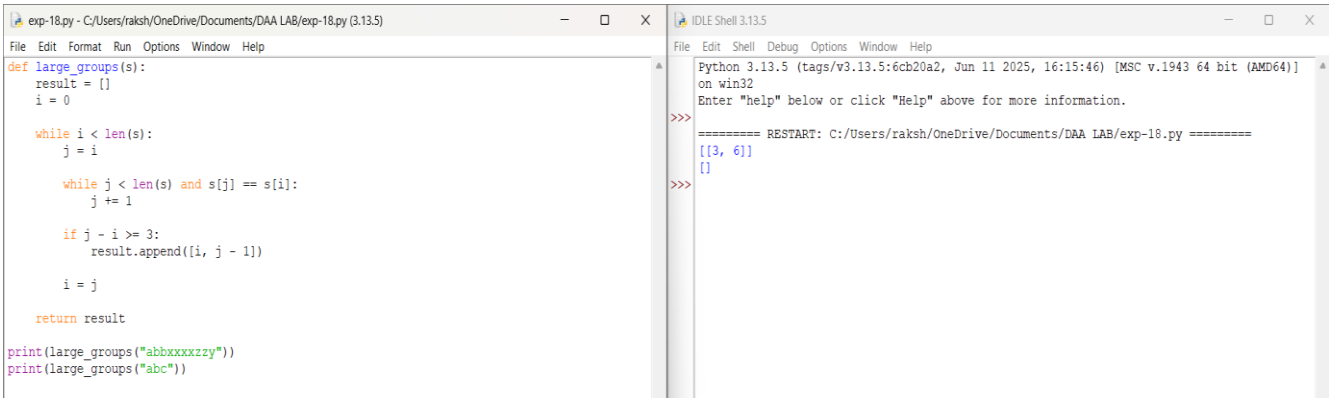
print(large_groups("abbxxxxzzy"))
print(large_groups("abc"))
```

Input:

```
 "abbxxxxzzy"
"abc"
```

Output:

```
[[3, 6]]
[]
```



Time Complexity: O(n)

Space Complexity: O(k)

Result:

The consecutive groups of three or more identical characters were identified successfully.

Experiment 19: Optimized Bubble Sort

Aim:

To modify Bubble Sort so that it stops early when the list becomes sorted.

Algorithm:

1. Read the input list.
2. Set a flag to indicate whether any swap occurs.
3. Compare adjacent elements and swap them if they are in the wrong order.
4. If no swap occurs during a complete pass, stop the algorithm.
5. Otherwise, repeat the passes until the list is sorted.
6. Display the sorted list.

Code (Python):

```
def bubble_sort(arr):
    arr = arr.copy()
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

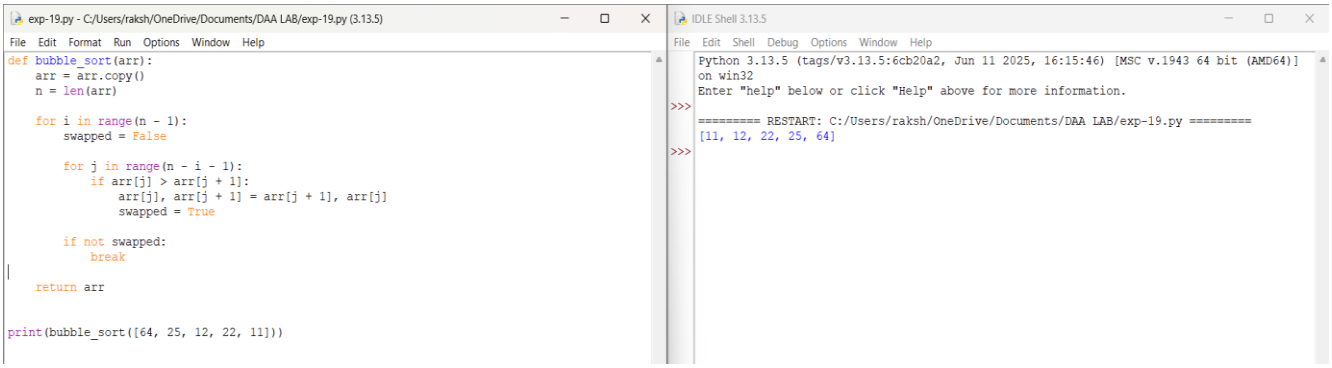

print(bubble_sort([64, 25, 12, 22, 11]))
```

Input:

[64, 25, 12, 22, 11]

Output:

[11, 12, 22, 25, 64]



Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Result:

The optimized Bubble Sort sorted the list and stopped early when no swaps were required.

Experiment 20: Testing Optimized Bubble Sort

Aim:

To test the optimized Bubble Sort function using different input lists.

Algorithm:

1. Define the optimized Bubble Sort function.
2. Provide the given test-case lists as input.
3. Sort each list using the function.
4. Compare each result with its expected output.
5. Display the sorted lists.

Code (Python):

```
def bubble_sort(arr):
    arr = arr.copy()
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr


test_cases = [
    [64, 25, 12, 22, 11],
    [29, 10, 14, 37, 13],
    [3, 5, 2, 1, 4],
    [1, 2, 3, 4, 5],
    [5, 4, 3, 2, 1]
]
```

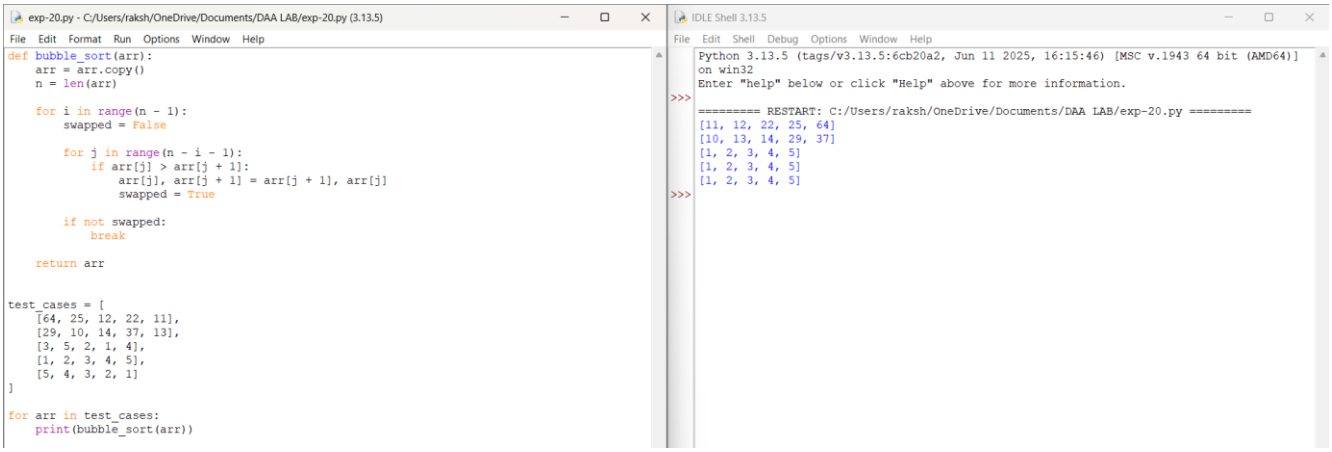
```
for arr in test_cases:
    print(bubble_sort(arr))
```

Input:

[64, 25, 12, 22, 11]
[29, 10, 14, 37, 13]
[3, 5, 2, 1, 4]
[1, 2, 3, 4, 5]
[5, 4, 3, 2, 1]

Output:

[11, 12, 22, 25, 64]
[10, 13, 14, 29, 37]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]



Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Result:

All five test cases produced the expected sorted output, including the already sorted and reverse-sorted lists.